CLAUDE CODE

Implementation Master Plan

AI Drug Compatibility Report — Pharmacogenomics Platform

Timeline	Phases	Tasks	Target
12 Days	**4 Phases**	**16 Tasks**	**Hackathon Win**

This document is the authoritative implementation guide for Claude Code.

It contains the complete project context, architecture decisions, data sources, API integrations, folder structure, and phase-by-phase task breakdown. Claude Code should treat this as the single source of truth and execute tasks in order without requiring further clarification on scope or tech choices.

1. Project Context & Background

1.1 What We Are Building

An AI-powered pharmacogenomics platform that allows users to upload their raw DNA data (from 23andMe or AncestryDNA), input their current medications, and receive a personalized drug compatibility report in under 60 seconds.

The core insight: 2 out of 5 drug prescriptions fail because doctors prescribe based on population averages, not individual genetics. Pharmacogenomics — the science of how genes affect drug response — exists and is proven, but is locked inside expensive labs ($500+) and requires a doctor referral. This platform makes it accessible, affordable, and instant.

One-Line Pitch

- Upload your DNA file. Enter your medications. Get a personalized report showing which drugs are safe, which are risky, and what alternatives exist — all powered by real pharmacogenomic science.

1.2 Why This Was Chosen (Hackathon Context)

This idea was selected over 4 other hackathon ideas after evaluating all on feasibility (12-day timeline), scalability, wow factor, and win probability. Here is why this won:

Criterion	Score	Reason
12-day feasibility	8/10	LLM + RAG + file parsing — no simulation engine needed
Scalability	9/10	SaaS model, $29/report or $99/yr, B2B hospital sales path
Wow factor	9.5/10	Live upload → 60-second report stuns judges every time
Market size	High	$50B pharmacogenomics market, <1% consumer penetration
Competition	Low	Labs charge $500+ with doctor referral. No consumer app exists

1.3 Hackathon Demo Strategy

The winning demo moment: the presenter uploads a real (publicly available) 23andMe sample file live on stage, enters 2-3 common medications, and the report appears within 60 seconds. Judges will immediately understand the value without any explanation.

Critical Demo Requirements

- Sub-60-second report generation from upload to rendered report

- A "Try Demo" button on the landing page preloaded with sample DNA + medications (no 23andMe account needed)
- PDF export of the report — judges and patients both need to share it with their doctor
- Medical disclaimer prominently displayed — judges in health/AI tracks specifically look for this
- Offline backup: a screen recording of a successful demo run, in case of network issues at venue

2. System Architecture

2.1 High-Level Architecture

The system has three layers: a Next.js frontend for user interaction, a FastAPI Python backend for all processing logic, and a PostgreSQL + pgvector database for the pharmacogenomics knowledge base.

Frontend (Next.js)	Backend API (FastAPI)	AI & Data Layer
User-facing React app. Handles file upload, medication input, report display, and PDF export.	Python backend. Parses 23andMe files, queries PharmGKB RAG, calls Claude API, assembles report JSON.	Claude API for report generation. pgvector for RAG. PharmGKB for drug-gene interaction data.

2.2 Full Tech Stack

Layer	Technology	Purpose	Notes
Frontend	Next.js 14 (App Router)	React framework	TypeScript, server components
Styling	Tailwind CSS	UI styling	Utility-first, fast to build
File Upload	react-dropzone	Drag-and-drop DNA file	Accepts .txt only
Animations	Framer Motion	Report reveal animations	Lightweight, no GSAP
PDF Export	jsPDF + html2canvas	Download report as PDF	Client-side, no server needed
Backend	FastAPI (Python 3.11)	REST API	Async, fast, typed with Pydantic
Data Validation	Pydantic v2	Request/response schemas	Auto-validates all inputs
DNA Parser	Custom Python module	Parse 23andMe .txt files	See Section 4.1
AI / LLM	Claude API (claude-sonnet)	Report generation	Structured JSON output
RAG	LangChain + pgvector	Retrieve drug-gene context	Embeddings at startup
Embeddings	OpenAI text-embedding-3-small	Vectorize PharmGKB records	Or use local model
Database	PostgreSQL 15 + pgvector	Knowledge base + vectors	Single DB, two purposes
Cache	Redis	Cache frequent drug lookups	Optional but recommended

Drug Names	RxNorm API (NLM)	Drug name normalization	Free, no API key needed
Deploy (FE)	Vercel	Frontend hosting	Free tier works for demo
Deploy (BE)	Railway	Backend + DB hosting	Free trial for demo
CI/CD	GitHub Actions	Auto-deploy on push	Simple workflow

2.3 Request Flow (End to End)

Understanding the full data flow is essential. Here is what happens from the moment the user uploads their DNA file to the moment the report renders:

1. User drags a 23andMe .txt file onto the upload zone and enters their medications.
2. Frontend sends a multipart/form-data POST to /api/analyze with the file and medication list.
3. FastAPI receives the request. The DNA parser reads the .txt file and extracts all rsIDs and genotypes.
4. The SNP filter keeps only pharmacogenomically relevant SNPs (CYP2D6, CYP2C19, CYP3A4, CYP2C9, VKORC1, SLCO1B1, DPYD, TPMT gene variants).
5. For each medication entered, the system normalizes the drug name via RxNorm API to get the standard name and RxCUI identifier.
6. The RAG retriever queries pgvector: given the users SNP profile + drug list, find the most relevant PharmGKB clinical annotations.
7. The assembled context (SNPs + annotations) is sent to the Claude API with a structured system prompt (see Section 5.2).
8. Claude returns a JSON report with per-drug compatibility scores, risk levels, explanations, and alternatives.
9. FastAPI validates the JSON with Pydantic and returns it to the frontend.
10. The React report page renders the results with color-coded cards. The user can download a PDF.

3. Project Folder Structure

Claude Code should create this exact folder structure. Do not deviate from it. The monorepo pattern keeps frontend and backend in one repository for easier deployment.

```
pharmgeno-ai/
|-- frontend/
|   |-- app/
|   |   |-- page.tsx                # Landing page
|   |   |-- upload/page.tsx         # Upload flow
|   |   |-- report/page.tsx         # Report display
|   |   |-- api/analyze/route.ts    # API proxy (optional)
|   |-- components/
|   |   |-- UploadZone.tsx          # Drag-and-drop DNA file upload
|   |   |-- MedicationInput.tsx     # Autocomplete medication entry
|   |   |-- ReportCard.tsx          # Per-drug compatibility card
|   |   |-- RiskBadge.tsx           # Green/Amber/Red badge
|   |   |-- OverallScore.tsx        # Big score display
|   |   |-- PDFExport.tsx           # Download report button
|   |   |-- Disclaimer.tsx          # Medical disclaimer banner
|   |-- lib/
|   |   |-- api.ts                  # Fetch wrappers to backend
|   |   |-- types.ts                # TypeScript interfaces
|   |-- public/
|   |   |-- sample-demo.txt         # Sample 23andMe file for demo mode
|   |-- tailwind.config.ts
|   |-- next.config.ts
|   |-- package.json
|-- backend/
|   |-- main.py                     # FastAPI app entry point
|   |-- api/
|   |   |-- routes.py               # All route definitions
|   |   |-- schemas.py              # Pydantic request/response models
|   |-- core/
|   |   |-- dna_parser.py           # 23andMe .txt file parser
|   |   |-- snp_filter.py           # Filter to PGx-relevant SNPs
|   |   |-- drug_normalizer.py      # RxNorm drug name lookup
|   |   |-- rag_retriever.py        # pgvector similarity search
|   |   |-- report_generator.py     # Claude API call + parsing
|   |-- db/
|   |   |-- database.py             # PostgreSQL connection (asyncpg)
|   |   |-- models.py               # SQLAlchemy ORM models
|   |   |-- seed_pharmgkb.py        # One-time DB seeder script
|   |-- data/
|   |   |-- pharmgkb/               # Raw PharmGKB TSV files
|   |   |-- rxnorm_cache.json       # Cached drug name lookups
|   |-- requirements.txt
|   |-- .env.example
|-- .github/workflows/deploy.yml    # CI/CD
|-- README.md
|-- docker-compose.yml              # Local dev (postgres + redis)
```

4. Data Sources & Setup

4.1 23andMe File Format

23andMe provides a raw data download as a .txt file. Claude Code must handle this exact format:

```
# This data file generated by 23andMe
# Sample File
#
# rsid         chromosome  position    genotype
rs4477212      1           82154        AA
rs3094315      1           752566       AG
rs3131972      1           752721       AG
rs12562034     1           768448       GG
rs11240777     1           798959       AG
...
```

Key parsing rules:

- Lines starting with # are comments — skip them
- Columns are tab-separated: rsid, chromosome, position, genotype
- Genotype can be 2 characters (diploid: AA, AG, GG) or -- for no-call
- File can be 600,000 to 1,000,000+ lines — use streaming/chunked reading, never load all into memory at once
- Only keep rsIDs that appear in our PGx SNP lookup table (see Section 4.3)

4.2 PharmGKB Database

PharmGKB is the gold-standard pharmacogenomics database, freely available at pharmgkb.org. These are the files to download and use:

File	URL Path	Use
clinical_annotations.tsv	/downloads/data/latestVersion/zip/clinicalAnnotations.zip	Drug-gene clinical evidence with dosing guidelines
var_drug_ann.tsv	/downloads/data/latestVersion/zip/variantAnnotations.zip	Specific variant-drug interaction annotations
genes.tsv	/downloads/data/latestVersion/zip/genes.zip	Gene metadata and HGNC IDs
chemicals.tsv	/downloads/data/latestVersion/zip/chemicals.zip	Drug/chemical metadata with RxNorm mappings

PharmGKB Setup Steps for Claude Code

- Download the ZIP files from pharmgkb.org/downloads (no account required)
- Extract into backend/data/pharmgkb/
- Run backend/db/seed_pharmgkb.py to parse TSVs and load into PostgreSQL
- The seeder also generates and stores vector embeddings for each annotation
- Total load time: ~5-10 minutes on first run. After that, DB is ready.

4.3 Priority PGx SNPs

Do not try to analyze all 1M+ SNPs in a 23andMe file. Focus only on the clinically actionable ones. These 5 gene families cover interactions for ~70% of commonly prescribed medications:

Gene	Key Variants (rsIDs)	Drugs Affected
CYP2D6	rs3892097, rs1065852, rs28371706	Codeine, tramadol, SSRIs, antipsychotics, beta-blockers
CYP2C19	rs4244285, rs4986893, rs28399504	Clopidogrel, PPIs, antidepressants, antifungals
CYP2C9	rs1799853, rs1057910	Warfarin, NSAIDs, phenytoin, glipizide
VKORC1	rs9923231	Warfarin dosing (critical!)
CYP3A4/5	rs776746, rs35599367	Statins, immunosuppressants, HIV drugs, many others
SLCO1B1	rs4149056	Simvastatin (myopathy risk)
DPYD	rs3918290, rs55886062	5-Fluorouracil, capecitabine (chemo toxicity)
TPMT	rs1800460, rs1142345	Azathioprine, mercaptopurine (chemo toxicity)

4.4 RxNorm Drug Normalization

Users will type medication names in many ways ("Lipitor", "atorvastatin", "atorvastatin calcium 20mg"). The RxNorm API normalizes these to standard names. It is free, no API key required.

```
# Example API call — no auth needed
GET https://rxnav.nlm.nih.gov/REST/rxcui.json?name=lipitor&search=1

# Returns:
{ "idGroup": { "rxnormId": ["41493"] } }

# Then get the standard name:
GET https://rxnav.nlm.nih.gov/REST/rxcui/41493/property.json?propName=RxNorm%20Name
# Returns: "atorvastatin"
```

Cache all RxNorm lookups in rxnorm_cache.json so repeat queries are instant and we do not hammer the API during the live demo.

5. AI Layer — Claude API Integration

5.1 RAG Pipeline

The RAG (Retrieval-Augmented Generation) pipeline is the core of the system. It ensures Claude generates reports based on real pharmacogenomic evidence rather than hallucinating.

Pipeline steps:

11. At app startup (or seeding), embed each PharmGKB clinical annotation using text-embedding-3-small. Store the embedding vector alongside the annotation in pgvector.
12. When a report is requested, construct a query string combining the users SNP profile and drug list. Example: "CYP2D6 poor metabolizer codeine tramadol warfarin"
13. Run a similarity search in pgvector to retrieve the top 15 most relevant annotations.
14. Format the retrieved annotations as structured context text.
15. Send context + user data to Claude API with the system prompt below.

5.2 Claude API Prompt

This is the exact system prompt to use. Do not change the structure — the JSON output schema must be strictly followed for the frontend to render correctly.

```
SYSTEM PROMPT:
"""
You are a clinical pharmacogenomics AI assistant. You analyze
a patients genetic variants and medications to generate a drug
compatibility report. You base your analysis ONLY on the
PharmGKB evidence provided in the context. Do not use general
knowledge outside of the provided context.

IMPORTANT: You must respond with ONLY valid JSON, no markdown,
no preamble, no explanation outside the JSON structure.

CRITICAL DISCLAIMER: Always include in your response that this
report is for informational purposes only and is not a substitute
for professional medical advice.

Output this exact JSON schema:
{
  "overall_risk_score": <integer 1-10, 10 = safest>,
  "summary": "<2-3 sentence plain English summary>",
  "disclaimer": "This report is for informational purposes only...",
  "drugs": [
    {
      "drug_name": "<standard drug name>",
      "compatibility": "<safe|caution|avoid>",
      "score": <integer 1-10>,
      "gene_variants_involved": ["<gene:variant>"],
      "explanation": "<plain English, max 3 sentences>",
```

```
      "clinical_significance": "<high|moderate|low>",
      "alternatives": ["<alternative drug name>"],
      "evidence_level": "<1A|1B|2A|2B|3>",
      "source": "PharmGKB"
    }
  ],
  "gene_summary": [
    { "gene": "<gene name>", "phenotype":
"<poor|intermediate|normal|rapid|ultrarapid metabolizer>", "clinical_impact": "<one
sentence>" }
  ]
}
"""

USER MESSAGE:
"""
Patient genetic profile:
<formatted SNP profile>

Current medications:
<normalized drug list>

Relevant PharmGKB annotations:
<RAG retrieved context>
"""
```

5.3 Claude API Configuration

Use these settings for the Claude API call:

```
model = "claude-sonnet-4-5"   # Good balance of speed and quality
max_tokens = 4096              # Enough for full multi-drug report
temperature = 0               # Zero temp for consistent structured output
stream = False                 # Parse complete JSON, not streamed
```

Handling Claude API Errors

- Always wrap the Claude API call in try/except
- If JSON parsing fails, retry once with a stricter prompt ("respond with ONLY JSON, nothing else")
- If retry fails, return a partial report with the raw text and a flag: "ai_parse_error": true
- Never crash the demo — always return something to the user
- Log all Claude responses to a file during development for debugging

6. Backend API Endpoints

6.1 Endpoint Reference

Method	Path	Input	Output	Notes
POST	/api/analyze	multipart: file + medications[]	ReportResponse JSON	Main endpoint — runs full pipeline
POST	/api/demo	medications[] only	ReportResponse JSON	Uses pre-loaded sample DNA file
GET	/api/drugs/search?q=	query string	Drug name suggestions[]	RxNorm autocomplete proxy
GET	/api/health	none	{ status: ok }	Health check for deployment

6.2 Pydantic Schemas

Define these schemas in backend/api/schemas.py. These enforce input validation and output structure:

```
from pydantic import BaseModel
from typing import Literal, Optional

class AnalyzeRequest(BaseModel):
    medications: list[str]   # Raw medication names from user

class DrugReport(BaseModel):
    drug_name: str
    compatibility: Literal["safe", "caution", "avoid"]
    score: int
    gene_variants_involved: list[str]
    explanation: str
    clinical_significance: Literal["high", "moderate", "low"]
    alternatives: list[str]
    evidence_level: str
    source: str

class GeneSummary(BaseModel):
    gene: str
    phenotype: str
    clinical_impact: str

class ReportResponse(BaseModel):
    overall_risk_score: int
    summary: str
    disclaimer: str
```

```
    drugs: list[DrugReport]
    gene_summary: list[GeneSummary]
    processing_time_seconds: float   # Add this for the demo
```

7. Frontend Implementation

7.1 Page Flow

Three pages. Keep the flow simple — this is a hackathon, not a production SaaS:

Page	Route	Purpose	Key Components
Landing	/	Hero + pitch + CTA buttons	Value prop, "Try Demo" button, "Upload your file" button
Upload	/upload	File upload + medications	UploadZone, MedicationInput, analyze button
Report	/report	Full compatibility report	OverallScore, DrugCard x N, GeneSummary, PDFExport, Disclaimer

7.2 Key Component Specs

UploadZone.tsx

- Accept .txt files only (23andMe raw data format)
- Show file name and size after selection
- Validate: reject files under 10KB (too small to be real) or over 50MB (malformed)
- Display a friendly error if wrong file type uploaded
- State: idle | dragging | selected | uploading | error

MedicationInput.tsx

- Text input with autocomplete calling /api/drugs/search
- Pills/chips display for added medications — click to remove
- Allow up to 10 medications
- Debounce search: 300ms after last keystroke
- Show generic name in autocomplete (not brand name) for clarity

ReportCard.tsx

- One card per drug in the report
- Header: drug name + RiskBadge (green=safe, amber=caution, red=avoid)
- Body: explanation text, gene variants involved, evidence level
- Footer: alternatives list (if any)
- Animate in with Framer Motion staggered delay (index * 0.1s)

Disclaimer.tsx

- Must be rendered at the TOP of the report page, above everything else
- Yellow/amber background, never dismissible

- Text: "This report is for informational purposes only and does not constitute medical advice. Always consult your doctor or pharmacist before making any changes to your medications."

7.3 Demo Mode

The landing page "Try Demo" button must work without the user having a 23andMe file. Implementation:

- Store a real sample 23andMe file at frontend/public/sample-demo.txt (grab from GitHub genomics repos)
- Demo button sends a POST to /api/demo with a pre-selected medication list: ["warfarin", "clopidogrel", "simvastatin"]
- The backend uses the stored sample DNA file for this endpoint
- This ensures judges can always see the full flow regardless of whether they have DNA data

7.4 PDF Export

Use jsPDF + html2canvas for client-side PDF generation:

- Capture the report div with html2canvas
- Add it to a jsPDF document with the patients name (if provided) and report date
- Include the disclaimer text as the first page
- Filename: pharmgeno-report-[date].pdf
- Do NOT use a server-side PDF endpoint — too slow and complex for a hackathon

8. Implementation Phases (12 Days)

1 Foundation & Data Layer — Days 1–3

Set up monorepo & dev environment

Init Next.js 14 frontend + FastAPI backend. Set up docker-compose.yml with PostgreSQL 15 + pgvector + Redis. Create .env.example with all required keys.

`[Next.js 14]` `[FastAPI]` `[Docker]` `[PostgreSQL]` `[pgvector]` `[Redis]`

Download & load PharmGKB database

Download clinical_annotations.tsv and var_drug_ann.tsv from pharmgkb.org. Write backend/db/seed_pharmgkb.py to parse TSVs and insert into PostgreSQL. Verify row counts after seeding.

`[PharmGKB]` `[pandas]` `[PostgreSQL]` `[asyncpg]`

Build the 23andMe file parser

Write backend/core/dna_parser.py. Parse .txt format, skip comment lines, return dict of {rsid: genotype}. Handle edge cases: no-call (--), indel variants (I/D). Test against sample file.

`[Python]` `[pandas]` `[streaming file read]`

Build SNP filter & PGx lookup table

Write backend/core/snp_filter.py. Hard-code the PGx SNP list (Section 4.3). Filter parsed SNPs to only PGx-relevant ones. Map rsIDs to gene names and phenotype rules.

`[Python]` `[dict lookup]` `[phenotype mapping]`

Build RAG pipeline with pgvector

Write backend/core/rag_retriever.py. Embed all PharmGKB annotations with text-embedding-3-small at startup. Store in pgvector. Implement similarity_search() function that takes SNP profile + drugs, returns top 15 annotations.

`[LangChain]` `[pgvector]` `[OpenAI embeddings]`

2 AI Core & Report Engine — Days 4–6

Implement RxNorm drug normalization

Write backend/core/drug_normalizer.py. Call RxNorm REST API to normalize user-entered drug names. Cache results in rxnorm_cache.json. Handle failures gracefully (return original name if API fails).

`[RxNorm API]` `[httpx]` `[JSON cache]`

Build Claude API report generator

Write backend/core/report_generator.py. Assemble context string from SNP profile + RAG results. Call Claude API with system prompt from Section 5.2. Parse JSON response. Validate with Pydantic. Return ReportResponse.

`[Claude API]` `[anthropic SDK]` `[Pydantic]` `[JSON parsing]`

Wire up the /api/analyze endpoint

Connect all core modules in backend/api/routes.py. Chain: file upload -> dna_parser -> snp_filter -> drug_normalizer -> rag_retriever -> report_generator. Add timing (time.time() before and after). Return full ReportResponse with processing_time_seconds.

`[FastAPI]` `[async/await]` `[multipart upload]`

End-to-end integration test

Test the full pipeline with a real 23andMe sample file and 3 medications. Verify the JSON output matches ReportResponse schema. Check processing time is under 60 seconds. Fix any bottlenecks.

`[pytest]` `[httpx test client]` `[timing profiler]`

3 Frontend & UX

Days 7–9

Build landing page

Create app/page.tsx. Hero section with one-line pitch, "Upload your file" CTA, and "Try Demo" button. Add brief how-it-works section (3 steps). Mobile responsive.

`[Next.js]` `[Tailwind]` `[Framer Motion]`

Build upload page with file + medication input

Create app/upload/page.tsx with UploadZone and MedicationInput components. Validate inputs. Show loading state during analysis (progress bar + "Analyzing your genetic profile..." text). On success, redirect to /report with report data in state or localStorage.

`[react-dropzone]` `[Tailwind]` `[Next.js router]`

Build report display page

Create app/report/page.tsx. Render Disclaimer banner first. Then OverallScore (big number). Then staggered DrugCard components (Framer Motion). Then GeneSummary table. Then PDFExport button.

`[Framer Motion]` `[jsPDF]` `[html2canvas]` `[Tailwind]`

Implement demo mode + drug autocomplete

Add /api/demo endpoint in backend that uses sample DNA file. Wire frontend "Try Demo" button to this endpoint. Implement /api/drugs/search autocomplete endpoint with RxNorm. Add debounced autocomplete to MedicationInput.

`[Next.js API route]` `[RxNorm]` `[debounce]`

4 Polish, Deploy & Demo Prep

Days 10–12

Performance optimization — hit the 60-second target

Profile the full pipeline. Pre-embed all PharmGKB records once (not on each request). Add Redis caching for RxNorm lookups. Use FastAPI BackgroundTasks if needed. Measure and log processing_time_seconds for every request.

`[Redis]` `[async FastAPI]` `[cProfile]` `[timing logs]`

Error handling & edge cases

Handle: wrong file format, no PGx SNPs found in uploaded file, unknown drugs, Claude API timeout, empty report. Never crash on bad input. Add user-friendly error messages. Test all error paths manually.

`[FastAPI exception handlers]` `[React error boundary]`

Deploy to production

Deploy frontend to Vercel (connect GitHub repo, set env vars). Deploy backend to Railway (Dockerfile or Nixpacks auto-detect). Configure CORS in FastAPI to allow Vercel domain. Run full end-to-end test on production URLs.

`[Vercel]` `[Railway]` `[CORS]` `[env vars]`

Demo prep — pitch deck + rehearsal

Create 5-slide deck: (1) Problem, (2) The insight nobody acts on, (3) Live demo, (4) Market size + revenue model, (5) Team. Rehearse the live upload demo 5+ times. Record a backup screen recording. Test demo mode on a fresh browser (no cached data).

`[Canva/Figma]` `[screen recording]` `[OBS Studio]`

---

9. Environment Variables

Create backend/.env and frontend/.env.local with these variables. Never commit the actual values to git.

```
# backend/.env
ANTHROPIC_API_KEY=sk-ant-...
OPENAI_API_KEY=sk-...              # For text-embedding-3-small
DATABASE_URL=postgresql+asyncpg://user:pass@localhost:5432/pharmgeno
REDIS_URL=redis://localhost:6379
RXNORM_BASE_URL=https://rxnav.nlm.nih.gov/REST
DEMO_DNA_FILE_PATH=./data/sample-23andme.txt
MAX_FILE_SIZE_MB=50
REPORT_TIMEOUT_SECONDS=90

# frontend/.env.local
NEXT_PUBLIC_API_URL=http://localhost:8000   # Change to Railway URL in prod
```

10. Known Risks & Mitigations

These are the risks identified during planning. Claude Code should be aware of these and implement the mitigations proactively.

Risk	Likelihood	Mitigation
23andMe file too large to process in time	High	Stream the file line by line. Never load full 1M+ rows into memory. Stop reading once all PGx SNPs found.
Claude API returns malformed JSON	Medium	Use temperature=0. Retry once with stricter prompt. Fall back to partial report rather than crash.
RxNorm API slow or down during demo	Low	Pre-cache all medications for demo mode in rxnorm_cache.json. Demo mode bypasses live API calls.
PharmGKB data has no match for a drug	Medium	Return a "no data available" card for that drug. Never hallucinate — tell the user we lack data.
pgvector similarity search is slow	Low	Create an IVFFlat index on the embedding column after seeding. Tune nlist and nprobe parameters.
Medical accuracy concerns from judges	Medium	Lead with the disclaimer. Frame as "decision support" not "diagnosis". Cite PharmGKB evidence levels prominently.
Network outage during live demo	Low	Have demo mode cached locally. Have a screen recording backup ready. Practice the offline pitch.

11. Instructions for Claude Code

Read This Before Writing Any Code

- Work phase by phase. Complete Phase 1 fully before starting Phase 2.
- When setting up the database, run seed_pharmgkb.py and verify row counts before proceeding.
- The 23andMe parser is the riskiest piece of code — test it with the sample file immediately.
- Never hardcode API keys. Always read from environment variables.
- All FastAPI routes must be async. Use asyncpg for database, httpx for external API calls.
- The Claude API prompt in Section 5.2 must be used exactly as written. Do not paraphrase it.
- If a task is ambiguous, implement the simpler interpretation first. Speed matters more than perfection.
- After Phase 2, run the full pipeline end-to-end and verify processing_time_seconds < 60.
- Do not skip error handling. Every external API call must have a try/except.
- The Disclaimer component must be the first thing rendered on the report page — this is non-negotiable.

11.1 Quick Start Commands

Run these commands in order to get the project running locally:

```
# 1. Clone and setup
git init pharmgeno-ai && cd pharmgeno-ai

# 2. Start infrastructure
docker-compose up -d    # Starts PostgreSQL + pgvector + Redis

# 3. Backend setup
cd backend
python -m venv venv && source venv/bin/activate
pip install -r requirements.txt
cp .env.example .env    # Fill in your API keys
python db/seed_pharmgkb.py    # Load PharmGKB data (takes 5-10 mins)
uvicorn main:app --reload --port 8000

# 4. Frontend setup (new terminal)
cd frontend
npm install
cp .env.local.example .env.local
npm run dev    # Starts on localhost:3000

# 5. Test the full pipeline
curl -X POST http://localhost:8000/api/demo \
  -H "Content-Type: application/json" \
  -d '{"medications": ["warfarin", "clopidogrel", "simvastatin"]}'
```

End of Implementation Plan